

```
####dataset (https://www.kaggle.com/datasets/gpiosenska/sports-classification)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.models import load_model
from tensorflow.keras.utils import image_dataset_from_directory
from tensorflow.keras.applications import ResNet50V2
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.losses import CategoricalCrossentropy
from sklearn.metrics import classification_report

IMG_WIDTH = 256
IMG_HEIGHT = 256
BATCH_SIZE = 64

train_dataset_path="/content/drive/MyDrive/sportsimages/train"
validation_dataset_path="/content/drive/MyDrive/sportsimages/valid"
test_dataset="/content/drive/MyDrive/sportsimages/test"

train_datagen = ImageDataGenerator(rescale=1.0/255,
                                   zoom_range=0.1,
                                   width_shift_range=0.1,
                                   height_shift_range=0.1,
                                   fill_mode='nearest')

train_generator = train_datagen.flow_from_directory(train_dataset_path,
                                                    target_size=(IMG_WIDTH,
                                                                    IMG_HEIGHT),
                                                    batch_size=BATCH_SIZE,
                                                    class_mode='categorical',
                                                    )
```

```

shuffle=True)

validation_datagen = ImageDataGenerator(rescale=1.0/255)
validation_generator = validation_datagen.flow_from_directory(validation_d
ataset_path,

                                                                    target_size=(
IMG_WIDTH, IMG_HEIGHT),

                                                                    batch_size=BA
TCH_SIZE,

                                                                    class_mode='c
ategorical',

                                                                    shuffle=True)

test_datagen = ImageDataGenerator(rescale=1.0/255)

test_generator = test_datagen.flow_from_directory(test_dataset,
                                                    shuffle=False,
                                                    batch_size=BATCH_SIZE,
                                                    target_size = (IMG_WIDTH,
IMG_HEIGHT),

                                                    class_mode='categorical')

#create Model
resnet = ResNet50V2(
    input_shape=(256, 256, 3), include_top=False, weights="imagenet", clas
ses=100
)
resnet.trainable = False
model = Sequential()
model.add(resnet)
model.add(Flatten())
model.add(Dropout(0.1))
model.add(Dense(256, activation="relu"))
model.add(Dropout(0.1))
model.add(Dense(128, activation="relu"))
model.add(Dropout(0.1))
model.add(Dense(100, activation="softmax"))

model.compile(
    optimizer="adam", loss=CategoricalCrossentropy(), metrics=["accuracy"]
)

callback = EarlyStopping(monitor="loss", patience=2)
history = model.fit(

```

```

        train_generator,
        epochs=15,
        batch_size=32,
        callbacks=[callback],
        validation_data=validation_generator,
    )

#Plot Loss
plt.plot(history.history["loss"], 'o-',label="train")
plt.plot(history.history['val_loss'], 'o-',label="validation")
plt.title('Validation loss history')
plt.ylabel('Loss value')
plt.xlabel('No. epoch')
plt.legend()
plt.show()

# Plot Accuracy
plt.plot(history.history["accuracy"], 'o-',label="train")
plt.plot(history.history['val_accuracy'], 'o-',label="validation")
plt.title('Validation accuracy history')
plt.ylabel('Accuracy value (%)')
plt.xlabel('No. epoch')
plt.legend()
plt.show()

print(model.summary())

labels = {value: key for key, value in train_generator.class_indices.items()
()}

print("Label Mappings for classes present in the training and validation datasets\n")
for key, value in labels.items():
    print(f"{key} : {value}")

predictions = model.predict(test_generator)

y_pred = np.argmax(predictions, axis=1)
y_true = test_generator.classes

print(classification_report(y_true, y_pred, target_names=labels.values()))

```